

SISTEMA DE GESTIÓN LEGAL

API REST · Flask · PyMySQL · MySQL

LECCIONES APRENDIDAS

Taller Práctico de Base de Datos II — 2024

Aaron Fehrenbach 8-955-295

1. Introducción

Este documento recoge las lecciones aprendidas durante el desarrollo del taller práctico cuyo objetivo fue diseñar una base de datos relacional a partir de un prototipo de interfaz, implementarla en MySQL y exponerla mediante una API REST construida con Python, Flask y PyMySQL.

El sistema resultante gestiona expedientes legales, audiencias, clientes, aseguradoras, juzgados y usuarios, replicando la funcionalidad visual del prototipo de 'Agenda del Día'.

2. Stack Tecnológico Utilizado

Componente	Tecnología	Versión / Detalle
Base de datos	MySQL	8.x — Motor relacional principal
Backend / API	Python + Flask	Flask 3.x — Microframework web
Connector BD	PyMySQL	pip install pymysql

2. Diseño de la Base de Datos

2.1 Entidades y Tablas

El análisis del prototipo permitió identificar seis entidades principales. Cada tabla cuenta con una clave primaria auto-incremental, restricciones de unicidad donde corresponde y el campo created_at para auditoría.

Tabla	Propósito	Campos clave
aseguradoras	Catálogo de compañías aseguradoras	id, nombre (UNIQUE), telefono, email
juzgados	Catálogo de juzgados y autoridades	id, nombre (UNIQUE), ubicacion, tipo
usuarios	Abogados y personal del sistema	id, username (UNIQUE), password_hash, rol
clientes	Asegurados o partes particulares	id, nombre, cedula (UNIQUE)
expedientes	Caso legal principal — tabla central	id, numero_expediente, estado, claves foráneas
audiencias	Citas programadas por expediente y juzgado	id, expediente_id, juzgado_id, fecha, hora

2.2 Relaciones

- expedientes → clientes: relación N:1 (muchos expedientes por cliente).
- expedientes → aseguradoras: LEFT JOIN porque puede ser 'PARTICULAR' (sin aseguradora).
- expedientes → juzgados: relación N:1.
- expedientes → usuarios: cada expediente tiene un abogado asignado (usuario_asignado_id).
- audiencias → expedientes: un expediente puede tener múltiples audiencias (1:N).
- audiencias → juzgados: la audiencia ocurre en un juzgado específico.

2.3 Lecciones de Diseño

Las Buenas Prácticas:

- Usar LEFT JOIN en lugar de JOIN cuando la relación es opcional (aseguradora_id puede ser NULL en expedientes particulares). Un INNER JOIN habría excluido esos registros.
- Todos los campos de nombre en catálogos tienen la restricción NOT NULL UNIQUE, lo que previene duplicados a nivel de BD sin depender de la lógica de la aplicación.

Lecciones Aprendidas:

- MySQL requiere AUTO_INCREMENT (no AUTOINCREMENT como SQLite). Confundir ambos motores genera errores de sintaxis al migrar scripts entre sistemas.
- El campo password_hash en usuarios debe almacenar el hash de la contraseña, nunca el texto plano. En producción se recomienda usar bcrypt o pbkdf2 en lugar de SHA-256 simple.

3. Implementación de la API REST con Flask

3.1 Estructura del código (app_Taller_bd2.py)

El archivo principal sigue una estructura sencilla y directa:

```
from flask import Flask, jsonify
import pymysql

app = Flask(__name__)

def get_con():
    return pymysql.connect(
        host='localhost', user='root', password='...',
        database='taller1_clase_bd2', port=3306,
        charset='utf8mb4',
        cursorclass=pymysql.cursors.DictCursor
    )
```

3.2 Endpoints implementados

Método	Ruta	Descripción	Tabla(s) involucrada(s)
GET	/usuarios	Lista abogados y personal	usuarios
GET	/aseguradoras	Catálogo de aseguradoras	aseguradoras
GET	/juzgados	Catálogo de juzgados	juzgados
GET	/clientes	Lista de clientes/asegurados	clientes
GET	/expedientes	Expedientes con JOIN completo	expedientes + 4 tablas

3.3 Consulta con múltiples JOINS

El endpoint más complejo es /expedientes, que une cinco tablas para devolver información enriquecida en una sola respuesta:

```
SELECT
    e.id, e.numero_expediente,
    c.nombre AS cliente,
    a.nombre AS aseguradora,
    j.nombre AS juzgado,
    u.nombre_completo AS abogado,
    e.estado, e.descripcion,
    e.fecha_apertura, e.fecha_cierre
FROM expedientes e
JOIN clientes c ON e.cliente_id = c.id
LEFT JOIN aseguradoras a ON e.aseguradora_id = a.id
```

```
LEFT JOIN juzgados      j ON e.juzgado_id      = j.id
JOIN usuarios          u ON e.usuario_asignado_id = u.id
```

3.4 Lecciones de la API

Las Buenas Prácticas:

- Usar `cursorclass=pymysql.cursors.DictCursor` en la conexión hace que cada fila se devuelva como un diccionario Python `{columna: valor}`, lo cual `jsonify()` serializa directamente a JSON sin conversión manual.
- Usar alias con AS en los SELECT de JOINs (AS cliente, AS aseguradora) evita colisiones de nombres cuando varias tablas tienen una columna 'nombre', y hace el JSON resultante más legible.

Lecciones Aprendidas:

- La función `get_con()` abre una nueva conexión en cada petición. En producción esto puede agotar las conexiones disponibles del servidor bajo carga alta. La solución es usar un pool de conexiones (ej. `flask_mysqldb` o `SQLAlchemy` con `pool_size`).
- Si la BD contiene columnas de tipo DATE, TIME o DATETIME, PyMySQL las devuelve como objetos Python (`datetime.date`, `datetime.timedelta`) que `jsonify()` no puede serializar directamente. Se debe convertir a string: `str(row['fecha'])`.

4. Resumen de Lecciones Aprendidas

4.1 Base de Datos

#	Lección	Impacto
1	AUTO_INCREMENT es sintaxis MySQL; AUTOINCREMENT es SQLite. No son intercambiables.	Alto — rompe el script SQL
2	Usar LEFT JOIN cuando una FK puede ser NULL; INNER JOIN excluye esos registros.	Alto — pérdida de datos en respuesta
3	UNIQUE en columnas de catálogo previene duplicados a nivel de motor, no solo en la app.	Medio — integridad de datos
4	created_at y updated_at deben estar en todas las tablas para auditoría y depuración.	Medio — trazabilidad
5	Las contraseñas deben almacenarse como hash; nunca en texto plano.	Alto — seguridad crítica

4.2 API REST / Flask / PyMySQL

#	Lección	Impacto
1	DictCursor elimina la necesidad de mapear manualmente columnas a diccionarios.	Alto — simplifica el código
2	jsonify() no serializa datetime.date ni datetime.timedelta de PyMySQL; convertir a string.	Alto — error 500 en producción
3	Una conexión nueva por petición escala mal; usar pool de conexiones en producción.	Medio — rendimiento
4	Siempre cerrar cursor y conexión en un bloque finally o con context manager.	Medio — fugas de recursos
5	Los alias AS en JOINS son esenciales cuando múltiples tablas tienen columnas con igual nombre.	Alto — JSON incorrecto sin alias
6	debug=True solo debe usarse en desarrollo; expone información sensible del servidor.	Alto — seguridad

5. Conclusión

El taller demostró exitosamente el flujo completo desde un prototipo visual hasta una API funcional: análisis de pantallas, modelado relacional, implementación en MySQL y exposición de datos mediante Flask y PyMySQL.

Las lecciones más valiosas fueron: la importancia de usar DictCursor para simplificar la serialización JSON, el uso correcto de LEFT JOIN para relaciones opcionales, y la necesidad de gestionar adecuadamente las conexiones a la base de datos.

Este proyecto sienta las bases para escalar hacia un sistema de producción añadiendo autenticación, validación de datos, manejo de errores robusto y una capa de pool de conexiones.